Required certificates for mixes and covers

Human Compression project

Measurement representation of the accepted L016 edition

These two complete programs certify the coprime mix counterexample and the order-96 universal marked presentation. They are finite mathematical premises, using GAP's exact presentation algorithms. The two whole raw files are an alternative representation, not an additional charge. Optional validation, replay drivers and partial Lean evidence are inventoried separately. No measurement factor is asserted here.

Coprime mix certificate

```
# Literal finite certificate for the coprime-label counterexample.
# GAP subgroup arithmetic is the retained computational kernel.
Need := function(ok, msg)
  if not ok then Error(msg); fi;
end;
CheckIP := function(marks)
  local subs, mask, i, j, common, mask2;
  subs := [];
  for mask in [0..15] do
    Add(subs, Group(Filtered(marks, x -> QuoInt(mask,2^(Position(marks,x)-1)) mod 2 = 1), ()));
  od;
  for mask in [0..15] do
    for mask2 in [0..15] do
      common := Sum(Filtered([0..3], i -> QuoInt(mask,2^i) mod 2=1 and QuoInt(mask2,2^i) mod 2=1), i ->
      2^i);
      Need(Intersection(subs[mask+1],subs[mask2+1])=subs[common+1], "input intersection property");
    od;
  od;
  return subs;
end;
a := [(1,2)(3,5)(4,7)(6,8), (1,3)(2,4)(5,7)(6,9),
      (3,6)(4,7)(5,8)(9,10), (1,3)(2,5)(4,7)(10,11)];;
b := [(12,13),(13,14),(14,15),(15,16)];;
for marks in [a,b] do
  Need(ForAll(marks, x -> Order(x)=2), "nonidentity involutions");
  for i in [1..4] do
    for j in [i+2..4] do
      if j<=4 then Need(marks[i]*marks[j]=marks[j]*marks[i], "string commutation"); fi;
    od;
  od;
od;
G := Group(a);; S := Group(b);;
Need(Size(G)=660 and IsSimpleGroup(G), "literal simple 660-group");
Need(Size(S)=120, "literal simplex S5");
Need(List([1..3], i -> Order(a[i]*a[i+1]))=[3,5,3], "first type");
Need(List([1..3], i -> Order(b[i]*b[i+1]))=[3,3,3], "second type");
sa:=CheckIP(a);;sb:=CheckIP(b);;
Need(Size(sa[8])=60 and Size(sa[15])=60 and IsSimpleGroup(sa[8]) and IsSimpleGroup(sa[15]), "first A5
ends");
Need(Size(sb[8])=24 and Size(sb[15])=24, "second S4 ends");
Need(ForAll(b,x->SignPerm(x)=-1), "second orientation character");
m:=List([1..4],i->a[i]*b[i]);;
M:=Group(m);;A:=Group(m{[1..3]});;B:=Group(m{[2..4]});;C:=Group(m{[2,3]});;
Need([Size(M),Size(A),Size(B),Size(C),Size(Intersection(A,B))]=[79200,1440,1440,30,60], "exact mixed
orders");
Need(a[2] in A and a[2] in B and not(a[2] in C), "literal mixed intersection witness");
Print("COPRIME_CERTIFICATE_PASS\n");
Print("input_IP_checks=512; input_orders=660,120; types=3,5,3/3,3,3\n");
Print("mixed_orders=79200,1440,1440,30,60; witness=a1; second_orientable=true\n");
QUIT_GAP(0);
```

Source-base presentation certificate

```
# The single required finite presentation identification, extracted from the
# accepted Topic E universal row [4,3,3]/[3,4,3]. Optional finite validations
# are in separate complete programs, not premises of the uniform proof.
Need:=function(b,m) if not b then Error(m);fi;end;
Vects:=Tuples([0,1],4);
PermOf:=f->PermList(List(Vects,v->Position(Vects,f(v))));
r0:=PermOf(v->[(v[1]+1)mod 2,v[2],v[3],v[4]]);
r1:=PermOf(v->[v[1],(v[1]+v[2])mod 2,v[3],(v[3]+v[4])mod 2]);
r2:=PermOf(v->[(v[1]+v[2])mod 2,v[2],(v[3]+v[4])mod 2,v[4]]);
r3:=PermOf(v->[v[1],v[2],v[3],(v[4]+1)mod 2]);
rr:=[r0,r1,r2,r3];G:=Group(rr);Need(Size(G)=96,"explicit base order");
F:=FreeGroup(4);x:=GeneratorsOfGroup(F);
rels:=Concatenation(List(x,a->a^2),[(x[1]*x[2])^4,(x[2]*x[3])^3,
 (x[3]*x[4])^4,(x[1]*x[3])^2,(x[1]*x[4])^2,(x[2]*x[4])^2,
 (x[1]*x[2]*x[3])^3,(x[2]*x[3]*x[4])^3]);
U:=F/rels;Need(Size(U)=96,"exact retained universal presentation");
h:=GroupHomomorphismByImages(U,G,GeneratorsOfGroup(U),rr);
Need(h<>fail and IsBijective(h),"actual marked source identification");
Print("SOURCE_BASE96_MARKED_IDENTIFICATION_PASS\n");
```